Emotion Detection from Text or Speech using Common Lisp AI

1. Title

Emotion Detection from Text or Speech using a Common Lisp AI System

2. Introduction

Emotion detection is an important area of Artificial Intelligence because it helps computers understand human feelings from language. In this project, a system is developed to detect emotions from user-provided text or speech input. The system predicts emotions such as **Joy, Sadness, Anger, Fear, Surprise, and Neutral**.

This project does not use any machine learning library or trained model. Instead, it uses classic AI techniques such as a **knowledge base**, **production rules**, **forward chaining**, **backward chaining**, and a **perceptron-style scoring system**. The main AI logic is implemented in **Common Lisp**, and a frontend interface is created using **HTML, CSS, and JavaScript**.

The system is explainable because it does not only show the predicted emotion, but also displays the reasoning steps used to reach the result.

3. Objectives

- To develop an AI-based emotion detection system.
- To detect emotions from text or speech input.
- To use classic AI concepts instead of machine learning libraries.
- To implement the main AI engine using Common Lisp.
- To create a simple and interactive frontend using HTML, CSS, and JavaScript.
- To show the reasoning process using forward chaining and backward chaining.
- To calculate emotion scores using a perceptron-style weighted scoring method.

4. Technologies Used

- **Common Lisp**: Used for the main AI engine.
- **HTML**: Used to create the structure of the frontend.
- **CSS**: Used to design the user interface.
- **JavaScript**: Used to make the frontend interactive.
- **Speech Recognition API**: Used to take speech input in supported browsers.

- **SBCL**: Used to run the Common Lisp program from the terminal.

5. AI Concepts Used

5.1 Knowledge Base

The knowledge base stores information about each emotion. For every emotion, it contains:

- Emotion keywords
- Phrase patterns
- Feature weights

For example, the emotion **Joy** contains words like:

```
happy, excited, great, love, awesome, wonderful
```

The emotion **Fear** contains words like:

```
scared, afraid, nervous, worried, anxious, panic
```

This knowledge base is used by the inference system to identify possible emotions in the input sentence.

5.2 Forward Chaining

Forward chaining is a reasoning method that starts from known facts and applies rules to reach conclusions.

In this project, the known fact is the user input. The system checks the input for emotion-related words and patterns. If a match is found, a rule is fired.

Example:

```
Input: I am nervous about the presentation.
Fact: nervous is present
Rule: IF nervous is present THEN Fear rule fires
Conclusion: Fear has supporting evidence
```

Forward chaining helps the system generate visible reasoning facts.

5.3 Backward Chaining

Backward chaining starts with a goal and checks whether the available evidence supports that goal.

In this project, after the system predicts an emotion, backward chaining checks whether the predicted emotion is supported by:

- Matching keywords
- Matching phrase patterns
- Perceptron confidence score

Example:

```
Goal: Prove the emotion is Joy
Check: Does the input contain joy-related words?
Check: Does the perceptron score support Joy?
Conclusion: Joy is supported
```

Backward chaining helps explain why the final emotion was selected.

5.4 Perceptron-Style Scoring

A perceptron is a simple AI model that calculates output using input features and weights. In this project, a simplified perceptron-style scoring method is used.

The formula is:

```
score = base score + feature1 × weight1 + feature2 × weight2 + feature3 ×
weight3 ...
```

For each emotion, different features have different weights.

Example for Joy:

```
Joy score =
base score
+ positiveWords × 2.4
+ exclamation × 0.7
+ gratitude × 1.2
+ negativeWords × -1.2
```

If the input is:

```
I am excited and happy!
```

Then the system may extract:

```
positiveWords = 2
exclamation = 1
gratitude = 0
negativeWords = 0
```

So the Joy score becomes:

```
Joy score = 0.2 + 2 × 2.4 + 1 × 0.7
Joy score = 5.7
```

After calculating scores for all emotions, the system normalizes them into percentages. The emotion with the highest percentage is selected as the final prediction.

6. System Workflow

```
User enters text or uses microphone
        ↓
Input is sent to the system
        ↓
Text is tokenized into words
        ↓
Features are extracted
        ↓
Forward chaining checks rules
        ↓
Perceptron-style scoring calculates emotion scores
        ↓
Emotion with highest score is selected
        ↓
Backward chaining proves the selected emotion
        ↓
Final emotion, confidence, and explanation are displayed
```

7. Modules of the Project

7.1 Frontend Module

The frontend is built using HTML, CSS, and JavaScript. It contains:

- Text input box
- Microphone button

- Analyze button
- Sample button
- Clear button
- Emotion result display
- Confidence meter
- Perceptron score bars
- Forward chaining explanation
- Backward chaining explanation

7.2 Lisp AI Engine

The Lisp file contains the main AI implementation. Important functions include:

- `tokenize` : Converts input text into words.
- `extract-features` : Extracts useful features from the text.
- `run-forward-chaining` : Applies rule-based reasoning from facts.
- `perceptron-scores` : Calculates emotion scores.
- `choose-emotion` : Selects the emotion with the highest score.
- `prove-emotion` : Performs backward chaining.
- `analyze-emotion` : Runs the complete emotion detection process.
- `print-analysis` : Prints the result in the terminal.

7.3 JavaScript Demo Logic

The JavaScript file mirrors the Lisp AI logic so that the frontend can run directly in the browser. It updates the page dynamically after the user clicks the Analyze button or uses speech input.

8. Input and Output

Sample Input

```
I am excited but nervous about the presentation!
```

Extracted Evidence

```
excited → Joy
nervous → Fear
! → emotional intensity
```

Sample Output

```
Detected Emotion: Joy
Confidence: 49%
```

Explanation

The system predicts Joy because the word "excited" supports Joy and the exclamation mark increases emotional intensity. Fear also receives some score because of the word "nervous", but Joy receives the highest score.

9. Advantages

- Does not require a trained machine learning model.
- Uses explainable AI logic.
- Shows how the result was calculated.
- Demonstrates multiple AI concepts in one project.
- Supports both text and speech input.
- Can run in a browser and also through a Lisp terminal program.

10. Limitations

- The system depends on predefined keywords and patterns.
- It may not understand complex sarcasm or deep context.
- Accuracy depends on the quality of the knowledge base.
- Speech input depends on browser support.

11. Future Scope

- Add more emotions such as disgust, trust, and confusion.
- Improve the knowledge base with more words and phrases.
- Add support for multiple languages.
- Add voice tone analysis.
- Store previous results for comparison.
- Improve the frontend with animations and better visual feedback.

12. Conclusion

This project successfully demonstrates emotion detection using classic Artificial Intelligence techniques. The system takes text or speech input, extracts useful features, applies rule-

based reasoning, calculates emotion scores using a perceptron-style method, and predicts the final emotion.

The project is useful because it is explainable and educational. It clearly shows how AI concepts such as knowledge base, forward chaining, backward chaining, and perceptron scoring can be used together to solve a real-world problem. The Common Lisp engine shows the core AI logic, while the HTML, CSS, and JavaScript frontend makes the project interactive and easy to understand.

13. References

- Common Lisp programming concepts
- Artificial Intelligence rule-based systems
- Forward chaining and backward chaining inference methods
- Perceptron-based scoring concept
- Browser Speech Recognition API